SANTA CLARA UNIVERSITY	ECEN 511	Dylan Thornburg
Homework #1: Computer performance and MIPS datapath Answer the following problems and upload your answers to Camino in pdf foramt.		

1. [1 pt] Read the following articles and briefly describe how *chiplet* technology impacts each of the given design considerations:
   https://en.wikipedia.org/wiki/Chiplet
   https://www.arm.com/glossary/chiplet
   https://semiwiki.com/wikis/industry-wikis/chiplets-wiki/
   https://en.wikipedia.org/wiki/UCIe
   https://semiengineering.com/hbm-takes-on-a-much-bigger-role/
   https://www.imec-int.com/en/articles/chiplets-piecing-together-next-generation chips-part-i

   A. Manufacturing yield

   Increases yield. Smaller chiplets reduce the probability that a single defect will kill the whole chip, and thus less silicon is wasted. Chiplets can also be tested individually before assembly, which also improves yield.

   B. Time-to-market pressure

   Helps decrease pressure. Certain Chiplets are reusable for different SoCs, which allows less testing to be done (a chiplet may already be validated for a new chip) and parallel development.

   C. Communication latency/bandwidth (between components)

   Increases latency while requiring high bandwidth. Chiplets will have to be interconnected with buses. Although there have been advances to speed this up, it's simply not as fast as a fully monolithic chip. Extra care needs to be taken for chiplet designs such as optimal layout pathways to ensure good performance.

   D. Design flexibility

   Allows designs to be more flexible. Having multiple different chiplets that you can mix and match without changing the whole chip obviously makes changing things easier.

   E. Interoperability of IC with DRAM

   Chiplet designs allow for the integration of HBM more easily than their monolithic counterparts. The memory is stacked right by the chiplets, and as a result, it is much more efficient to utilize HBM on a chiplet design.

2. [3 pts] Suppose you wish to run a program P with 7.5 * 10^9instructions on a 5 GHz machine with a CPI of 0.8.

a. What is the expected CPU time?

CPU Time = Instr Count * CPI * Clock Cycle Time

= 7.5x10^9 * .8 * 1/(5x10^9)

=1.2s

b. When you run P, it takes 3 seconds of wall clock time to complete. What is the percentage of the CPU time P received?

1.2 / 3 = .4

40%

3. [3 pts] Consider two different implementations, P1 and P2, of the same instruction set. There are five classes of instructions (A, B, C, D, and E) in the instruction set. P1 has a clock rate of 4 GHz. P2 has a clock rate of 6 GHz. The average number of cycles for each instruction class for P1 and P2 is as follows:

Class	CPI on P1	CPI on P2
A	1	2
B	2	2
C	3	2
D	4	4
E	3	5

If the number of instructions executed in a certain program is divided equally among the classes of instructions except for class A, which occurs twice as often as each of the others, how much faster is P2 than P1?

For this program:

P1 CPI = ⅓ * (1) + ⅙ * (2+3+4+3) = 2.333

P2 CPI = ⅓ * (2) + ⅙ * (2+2+4+5) = 2.833

2.333/4ghz = 5.833x10^-10

2.833/6ghz = 4.722x10^-10

5.833/4.722 = 1.235, so about 23.5% faster

4. [3 pts] Describe the effect that a single stuck-at-0 fault (i.e., regardless of what it should be, the signal is always 0) would have for the signals shown below in the single cycle datapath. Which instructions, if any, will not work correctly? Explain why. Consider each of the following faults separately:

a. RegWrite = 0

You cannot write to any registers. No R-type, I-Type, or J-Type instructions will work because they rely on writing to registers.

b. ALUop0 = 0

ALUop produces 00 when meant to be 01. For branch instructions, ALUop0 needs to be 1. This means branch instructions will not work (at least not how they're intended to) as the adder will be adding instead of subtracting.

c. ALUop1 = 0

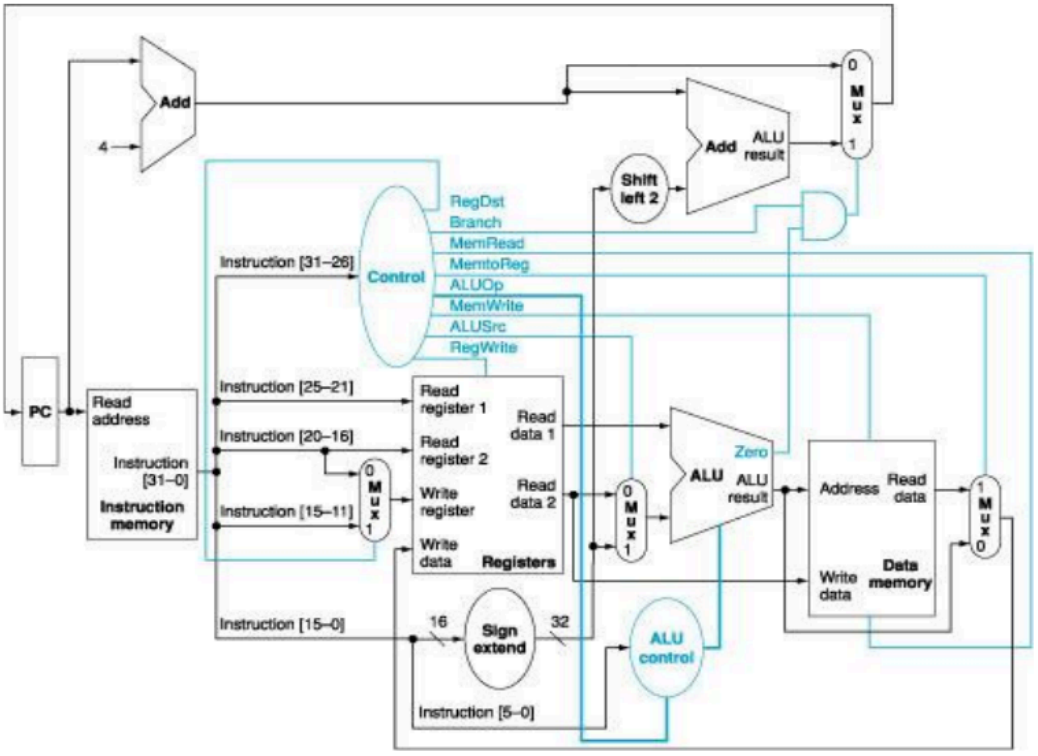
ALUop produces 00 when meant to be 10. This controls our R-type instructions. Now, any R-type instruction that requires subtraction will only add.

d. Branch = 0

The "and" gate with Branch is always 0. We cannot branch under any conditions now. We simply always increment the PC by 4.

e. MemRead = 0

The control to read from memory is always 0. Memtoreg still works so we DO still load data for load instructions, but the data is either 0 or X because we don't actually read from memory.

f. MemWrite = 0

The control to write to memory is always 0, and thus we never write to memory. Store instructions (S-Type in RISC-V) will not work.